

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 3 Exam Notes: Concurrent Processes

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Process Concept and Lifecycle States	2
1.1	Process Control Block (PCB)	2
1.2	Process States and Transitions	2
2	Process Generation & Scheduling Queues	2
2.1	Process Generation (<code>fork()</code> and <code>exec()</code>)	2
2.2	OS Schedulers	3
3	Inter-Process Communication (IPC)	3
4	Process Synchronization and Critical Section	3
4.1	The Critical Section Problem	3
4.2	Semaphores	3
4.3	Classical Synchronization Problems	4
4.3.1	Bounded-Buffer (Producer-Consumer) Problem	4
4.3.2	Readers-Writers Problem	4
4.3.3	Dining Philosophers Problem	4
4.4	Monitors	4

1. Process Concept and Lifecycle States

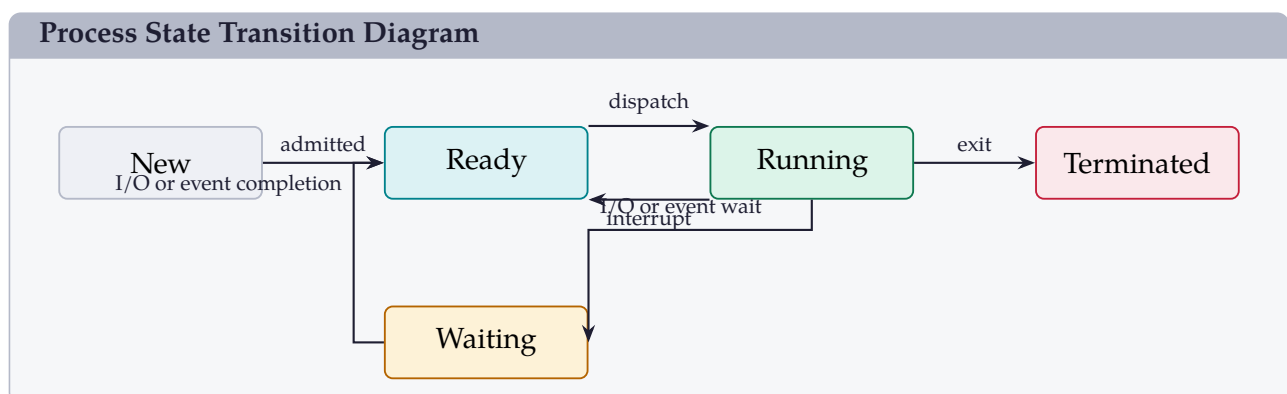
A **process** is a program in execution. It represents the unit of work in a modern time-sharing system.

1.1 Process Control Block (PCB)

Each process is represented in the operating system by a **Process Control Block (PCB)** (also called a task control block). The PCB contains:

- **Process State:** New, ready, running, waiting, or terminated.
- **Program Counter (PC):** Address of the next instruction to execute.
- **CPU Registers:** Accumulators, index registers, stack pointers.
- **CPU-Scheduling Information:** Process priority, pointers to scheduling queues.
- **Memory-Management Information:** Page tables or segment tables.
- **Accounting Information:** CPU time used, time limits.
- **I/O Status Information:** List of open files, allocated I/O devices.

1.2 Process States and Transitions



2. Process Generation & Scheduling Queues

2.1 Process Generation (`fork()` and `exec()`)

- **`fork()` System Call:** Creates a new, child process that is a duplicate of the parent process.
 - In the parent process, `fork()` returns the PID of the child.
 - In the child process, `fork()` returns 0.
 - If the call fails, it returns -1.
- **`exec()` System Call:** Replaces the process's memory space with a new executable program, changing the active program counter.

2.2 OS Schedulers

- **Long-Term Scheduler (Job Scheduler):** Selects processes from the pool (disk) and loads them into memory for execution. Controls the **degree of multiprogramming**.
- **Short-Term Scheduler (CPU Scheduler):** Selects from the processes in memory that are ready to execute and allocates the CPU to one of them. Executes extremely fast.
- **Medium-Term Scheduler:** Temporarily removes processes from memory (swapping) to reduce the degree of multiprogramming and free up RAM.

3. Inter-Process Communication (IPC)

Processes executing concurrently may be independent or cooperating. Cooperating processes require **Inter-Process Communication (IPC)**: 1. **Shared Memory:** A region of memory is established that cooperating processes can read and write to. Fast, but requires manual synchronization to prevent race conditions. 2. **Message Passing:** Processes exchange messages via the OS kernel without sharing memory space. Easier to implement but slower due to system calls.

4. Process Synchronization and Critical Section

A **race condition** occurs when multiple processes access and manipulate the same data concurrently, and the outcome of the execution depends on the particular order in which the access takes place.

4.1 The Critical Section Problem

The **Critical Section** is a code segment in which a process accesses shared resources.

Three Requirements for a Critical Section Solution

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can execute in their critical sections. 2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not executing in their remainder sections can participate in deciding which process enters next. 3. **Bounded Waiting:** There must be a limit on the number of times other processes can enter their critical sections after a process has made a request to enter, before that request is granted.

4.2 Semaphores

A **Semaphore** is an integer variable S that, apart from initialization, is accessed only through two standard atomic operations: `wait()` (or `P`) and `signal()` (or `V`).

Semaphore Atomic Implementations

```
void wait(Semaphore S) {
    while (S <= 0); // Busy wait (or block process in list)
    S--;
}

void signal(Semaphore S) {
    S++;
}
```

- **Binary Semaphore (Mutex):** Value can be either 0 or 1. Used for mutual exclusion.
- **Counting Semaphore:** Value ranges over an unrestricted integer domain. Used for resource allocation.

4.3 Classical Synchronization Problems

4.3.1 Bounded-Buffer (Producer-Consumer) Problem

Uses semaphores to coordinate a shared buffer of size N :

- `mutex` (binary, initial = 1): Protects the buffer insertion/removal.
- `empty` (counting, initial = N): Tracks empty buffer slots.
- `full` (counting, initial = 0): Tracks full buffer slots.

4.3.2 Readers-Writers Problem

Ensures multiple readers can access data concurrently, but only one writer can access it exclusively:

- `rw_mutex` (binary, initial = 1): Protects critical resource for writers.
- `mutex` (binary, initial = 1): Protects modification of `read_count` variable.

4.3.3 Dining Philosophers Problem

Represents resource allocation challenges among concurrent processes. 5 philosophers sit around a table with 5 chopsticks. Each needs 2 chopsticks to eat. Prone to deadlock if everyone picks up their left chopstick simultaneously.

4.4 Monitors

A **Monitor** is an abstract data type that encapsulates private data and procedures, ensuring that only one process can execute inside the monitor at any given time, preventing synchronization errors automatically.

Quick Revision — Module 3

Concurrent Processes | OE-EC604B

Producer-Consumer Semaphore Code

Producer Process:

```
do {
    // produce item
    wait(empty);
    wait(mutex);
    // add item to buffer
    signal(mutex);
    signal(full);
} while (true);
```

Consumer Process:

```
do {
    wait(full);
    wait(mutex);
    // remove item from buffer
    signal(mutex);
    signal(empty);
    // consume item
} while (true);
```

One-Line Definitions

- **Process:** A program in execution containing program counter, stack, data, and registers.
- **PCB:** Process Control Block; kernel data structure storing all attributes of a process.
- **Context Switch:** Saving CPU state of a process and restoring another to switch execution.
- **Race Condition:** State where multiple processes write/read shared data and outcome depends on execution order.
- **Critical Section:** Part of program accessing shared resource that must be protected.
- **Mutual Exclusion:** Preventing more than one process from entering a critical section.
- **Semaphore:** Protected integer variable used for signaling and process synchronization.
- **Monitors:** High-level programming construct providing automated mutual exclusion.

Frequently Asked / Viva Questions

Q1. Draw and label the 5-state process lifecycle model.

States: New → Ready → Running → Waiting → Terminated. Transitions: admitted, scheduler dispatch, interrupt, exit, I/O wait, I/O completion. (See TikZ block diagram).

Q2. What is the difference between fork() and exec() system calls?

Frequently Asked / Viva Questions (contd.)

`fork()` creates a new duplicate child process with its own PID, copying the parent's memory layout. `exec()` replaces the current process's entire memory space and code with a new binary program.

Q3. Describe the differences between Long-term, Medium-term, and Short-term schedulers.

Long-term: Controls degree of multiprogramming by loading jobs from disk to ready queue (RAM). Short-term: Allocates CPU to a ready process. Medium-term: Performs swapping of processes out of RAM to disk.

Q4. Define a race condition and explain how it can be prevented.

A race condition is when multiple processes access and write shared data concurrently and execution order dictates the final value. Prevented by enforcing mutual exclusion using synchronization mechanisms (semaphores, mutex).

Q5. Explain the three criteria that must be satisfied by any solution to the Critical Section problem.

1. Mutual Exclusion (only one process inside). 2. Progress (non-critical processes cannot block waiting ones). 3. Bounded Waiting (no starvation, waiting requests must be granted eventually).

Q6. Differentiate between Binary Semaphores and Counting Semaphores.

Binary semaphores take values 0 and 1 only, used primarily for mutual exclusion. Counting semaphores take any non-negative integer values, used to manage access to resources with multiple instances.

Q7. What is busy waiting? How do semaphores resolve it?

Busy waiting is when a process loops continuously checking a condition (`while(S <= 0);`), wasting CPU cycles. Resolved by having the semaphore block the process, putting it in a waiting queue and waking it up on `signal()`.

Q8. Describe the Dining Philosophers synchronization challenge.

5 philosophers share 5 chopsticks. If each picks up their left chopstick first, they will all wait indefinitely for their right chopstick, resulting in a deadlock.

Q9. What is a Monitor, and what advantage does it have over raw semaphores?

A monitor is a software package encapsulating shared variables and access methods. It ensures mutual exclusion automatically at compile-time/run-time, whereas semaphores are prone to programmer errors (like forgetting a `signal()` call).

Q10. Why does a context switch introduce performance overhead?

Because the CPU must save all register values, program counter, and memory maps of the current process into its PCB, and load those of the new process, during which no useful computation is done.

IPC Models Comparison

Feature	Shared Memory	Message Passing
Data Transfer Speed	Extremely Fast (RAM speeds)	Slower (system call overhead)
Kernel Involvement	Only during setup	For every message exchange
Synchronization	Managed by programmer (mutex)	Handled by OS kernel implicitly
Use Case	Large data volume, single machine	Small data volume, distributed systems